Lezione 6: Tipi induttivi, ricorsione e induzione

Stefano M. Nicoletti

1 Lezione 6: Tipi induttivi, ricorsione e induzione

1.1 Scopo della lezione

In questa lezione definiamo tipi con una struttura esplicita, funzioni ricorsive sui loro valori e dimostrazioni per induzione. Vediamo anche `rfl`, `unfold`, `rw` e `simp`.

1.2 Tipi induttivi

Nelle lezioni precedenti `Cosa : Type` lasciava aperta la struttura del dominio. Un tipo induttivo elenca invece i modi in cui possiamo costruire i suoi valori: questi modi si chiamano *costruttori*.

```
inductive Semaforo where
  | rosso
  | giallo
  | verde
```

I tre costruttori descrivono tutti i valori possibili di `Semaforo`. Possiamo definire un predicato mediante pattern matching:

```
def SiDeveFermare : Semaforo → Prop
  | .rosso => True
  | .giallo => True
  | .verde => False
```

Il pattern matching specifica il risultato per ciascun costruttore. Per ragionare su ogni semaforo, `cases s` apre i tre casi `rosso`, `giallo` e `verde`.

```
theorem lecture06_semaforo_ha_un_colore (s : Semaforo) :
    s = .rosso ∨ s = .giallo ∨ s = .verde := by
  cases s with
  | rosso =>
      apply Or.inl
      rfl
  | giallo =>
      apply Or.inr
      apply Or.inl
      rfl
  | verde =>
```

```
      apply Or.inr
      apply Or.inr
      rfl
```

1.3 I nostri numeri naturali

Definiamo un tipo di naturali senza usare `Nat`:

```
inductive Naturale where
  | zero
  | succ (n : Naturale)
```

```
open Naturale

def uno : Naturale := succ zero
```

```
open Naturale

def due : Naturale := succ uno
```

```
open Naturale

def tre : Naturale := succ due
```

`zero` è un naturale. Se `n` è un naturale, `succ n` è un naturale. Ogni valore finito di `Naturale` è una catena di `succ` che termina con `zero`.

1.4 Ricorsione

Una definizione ricorsiva risolve un problema usando il risultato dello stesso problema su un dato più semplice. Per `Naturale` distinguiamo il caso `zero` e il caso `succ n`.

```
open Naturale

def somma (m n : Naturale) : Naturale :=
  match n with
  | zero => m
  | succ n => succ (somma m n)
```

```
open Naturale

def doppio (n : Naturale) : Naturale :=
  match n with
  | zero => zero
  | succ n => succ (succ (doppio n))
```

`match n with` indica che `somma` analizza il secondo argomento. Nel caso `succ n`, la chiamata ricorsiva procede sul naturale più piccolo `n`. Lean accetta la definizione perché ogni chiamata ricorsiva si avvicina a un caso base.

1.5 Computazione, ‘rfl‘ e ‘unfold‘

Due espressioni sono definizionalmente uguali quando Lean può ridurle fino alla stessa espressione. In questi casi `rfl` chiude il goal:

```
open Naturale

example : somma uno uno = due := by
  rfl
```

```
open Naturale

theorem lecture06_somma_zero_destra (m : Naturale) :
    somma m zero = m := by
  rfl
```

```
open Naturale

theorem lecture06_somma_succ_destra (m n : Naturale) :
    somma m (succ n) = succ (somma m n) := by
  rfl
```

```
open Naturale

theorem lecture06_doppio_succ (n : Naturale) :
    doppio (succ n) = succ (succ (doppio n)) := by
  rfl
```

`unfold` rende esplicitamente visibile una definizione nel goal:

```
open Naturale

example : doppio uno = due := by
  unfold doppio
  unfold uno due
  rfl
```

1.6 Riscrivere con ‘rw‘

Se abbiamo `h : a = b`, `rw [h]` sostituisce `a` con `b` nel goal. `rw [← h]` usa la stessa uguaglianza nella direzione opposta.

```
open Naturale
```

```
theorem lecture06_rw_avanti
    (a b : Naturale)
    (h : a = b) :
    somma a zero = b := by
  rw [h]
  rfl
```

Dopo la riscrittura il lato sinistro è `somma b zero`, che si riduce a `b`. Nel verso opposto, se `h : a = b`, `rw [← h]` trasforma `somma b zero = a` in `somma a zero = a`.

```
open Naturale

theorem lecture06_rw_indietro
    (a b : Naturale)
    (h : a = b) :
    somma b zero = a := by
  rw ←[ h]
  rfl
```

1.7 Induzione

`somma m zero = m` si riduce subito: il secondo argomento è esplicitamente `zero`. Nell'enunciato `somma zero n = n`, invece, `n` è una variabile. Lean non sa ancora se `n` sia `zero` oppure `succ k`, e non può scegliere una riga della definizione di `somma`.

Non basta controllare alcuni valori, perché il risultato deve valere per tutti i naturali. Una dimostrazione per induzione ha un *caso base* per `zero` e un *passo induttivo*: assumiamo il risultato per `n` e lo dimostriamo per `succ n`. L'assunzione del passo è l'*ipotesi induttiva*.

Nel passo usiamo `rewrite [ih]`. Come `rw [ih]`, sostituisce con l'ipotesi induttiva. Qui però lascia visibile il goal finale, che chiudiamo esplicitamente con `rfl`. `rw` può chiudere da solo un goal che, dopo la riscrittura, ha la forma `t = t`.

```
open Naturale

theorem lecture06_zero_somma (n : Naturale) :
    somma zero n = n := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold somma
      rewrite [ih]
      rfl
```

Scegliamo la variabile d'induzione osservando la definizione ricorsiva. Qui `somma` analizza il secondo argomento, quindi nel goal `somma zero n = n` facciamo induzione su `n`.

La stessa scelta vale per un goal con due argomenti, come `somma (succ m) n = succ (somma m n)`: il secondo argomento di entrambe le somme è `n`, quindi facciamo induzione su `n`.

```
open Naturale

theorem lecture06_succ_somma (m n : Naturale) :
    somma (succ m) n = succ (somma m n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold somma
      rewrite [ih]
      rfl
```

1.8 Associatività, commutatività e ‘simp‘

Per l’associatività scegliamo `c`, il secondo argomento della somma esterna:

```
open Naturale

theorem lecture06_somma_assoc (a b c : Naturale) :
    somma (somma a b) c = somma a (somma b c) := by
  induction c with
  | zero =>
      rfl
  | succ c ih =>
      rw [lecture06_somma_succ_destra]
      rw [lecture06_somma_succ_destra]
      rw [lecture06_somma_succ_destra]
      rw [ih]
```

La commutatività non segue soltanto dalla computazione: `somma` non tratta simmetricamente i due argomenti. Nel caso base usiamo `zero_somma`. Nel passo usiamo `succ_somma` per spostare il successore dal primo argomento.

```
open Naturale

theorem lecture06_somma_comm (a b : Naturale) :
    somma a b = somma b a := by
  induction b with
  | zero =>
      rw [lecture06_zero_somma]
      rfl
  | succ b ih =>
      rw [lecture06_somma_succ_destra]
      rw [lecture06_succ_somma]
      rw [ih]
```

`simp` automatizza riscritture orientate verso forme più semplici. `simp only [lemmi]` limita esplicitamente le regole usate. `simp [lemma]` aggiunge una regola per quel comando. `attribute [local simp] lemma` la rende disponibile nella sezione corrente.

```
open Naturale

theorem lecture06_zero_somma_con_rw (n : Naturale) : somma zero n = n := by
  rw [lecture06_zero_somma]
```

```
open Naturale

theorem lecture06_zero_somma_con_simp_only (n : Naturale) : somma zero n = n
   := by
  simp only [lecture06_zero_somma]
```

```
open Naturale

theorem lecture06_zero_somma_con_simp (n : Naturale) : somma zero n = n := by
  simp [lecture06_zero_somma]
```

```
open Naturale

section SemplificazioneLocale

attribute [local simp] lecture06_zero_somma lecture06_succ_somma

theorem lecture06_succ_somma_con_simp (m n : Naturale) :
    somma (succ m) n = succ (somma m n) := by
  simp

end SemplificazioneLocale
```

```
open Naturale

@[simp] theorem lecture06_doppio_zero :
    doppio zero = zero := by
  rfl
```

`@[simp]` registra una regola per il resto del modulo. Lo usiamo per una regola canonica come `doppio zero = zero`. Associatività e commutatività non sono regole adatte al simp-set: cambiano soltanto il raggruppamento o l'ordine degli argomenti.

1.9 Una dimostrazione conclusiva

Combiniamo induzione e riscritture per dimostrare che il doppio di una somma è la somma dei doppi. La sottoespressione che resta bloccata è `somma m n`, perciò facciamo induzione su `n`.

```
open Naturale

theorem lecture06_doppio_somma (m n : Naturale) :
    doppio (somma m n) = somma (doppio m) (doppio n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      rw [lecture06_somma_succ_destra]
      rw [lecture06_doppio_succ]
      rw [lecture06_doppio_succ]
      rw [lecture06_somma_succ_destra]
      rw [lecture06_somma_succ_destra]
      rw [ih]
```

Dopo avere verificato ogni passaggio, possiamo esprimere la stessa dimostrazione con un insieme controllato di regole di semplificazione:

```
open Naturale

theorem lecture06_doppio_somma_breve (m n : Naturale) :
    doppio (somma m n) = somma (doppio m) (doppio n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      simp only [lecture06_somma_succ_destra, lecture06_doppio_succ, ih]
```

1.10 Strategia operativa

1. Individuiamo i costruttori del tipo. 2. Leggiamo quale argomento è analizzato dalla definizione. 3. Proviamo se il goal si riduce con `rfl`. 4. Usiamo con `rw` le uguaglianze già dimostrate. 5. Se resta una variabile nell'argomento analizzato, scegliamo quella variabile per l'induzione. 6. Automatizziamo con `simp only` o `simp`.

1.11 Fonti

- Lean developers, *Theorem Proving in Lean 4*, «Inductive Types»: https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- Lean developers, *Theorem Proving in Lean 4*, «Induction and Recursion»: https://lean-lang.org/theorem_proving_in_lean4/Induction-and-Recursion/.
- Lean developers, *Functional Programming in Lean*, «Datatypes and Patterns»: https://lean-lang.org/functional_programming_in_lean/Getting-to-Know-Lean/Datatypes-and-Patterns/.
- Lean developers, *The Lean Language Reference*, «Recursive Definitions»: https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/.

- Lean community, *Natural Number Game*: https://adam.math.hhu.de/.